ARDUINO OPTABLUEPRINT ECOSYSTEM

Library used to connect OPTA Controller and OPTA Expansion Modules on I2C and UART. This library discovers the presence of different Expansions (OPTA Digital, OPTA Analog, …) present on the I2C bus, automatically assign addresses to them and allow to control their Input/Output from the Opta Controller.

OPTA_myRTU project

Table of Contents

1 INTRODUCTION....6
1.1 Hardware....6
1.1.1 OPTA AUX Port....6
1.1.2 I2C bus....7
1.1.3 Switching Power Supply....7
1.2 Assign Addresses Process....8
1.2.1 Purposes of the Assign Addresses Process....8
1.2.2 PIN Usage....8
1.2.3 RGB LED Usage....9
1.2.4 I2C Assign Address Process....9
1.2.5 I2C Addresses for 1 to 5 Expansions....11
1.2.6 OptaModuleProtocol.cpp....12
1.3 About Classes....15
2 OptaBluePrint PROTOCOL....17
2.1 Header....17
2.1.1 Commands....17
2.1.2 Arguments....17
2.2 Payload....17
2.3 CRC....18
2.4 Messages Unhandled....18
2.5 Messages....18
2.5.1 RESET EXPANSION I2C ADDRESS....19
2.5.2 SET I2C ADDRESS....20
2.5.3 GET I2C ADDRESS AND TYPE....21
2.5.4 OD GET DIGITAL VALUES....22
2.5.5 OD GET ANALOG VALUES....23
2.5.6 OD GET ANALOG ALL VALUES....24
2.5.7 OD SET DIGITAL OUTPUTS....25
2.5.8 OD SET DEFAULT OUTPUT VALUES AND TIMEOUT....26
2.5.9 OA SET CHANNEL AS ADC....27
2.5.10 OA GET ADC VALUE....28

2.5.11 OA GET ALL ADC VALUE AT ONCE..... 29
2.5.12 OA SET CHANNEL AS DAC..... 30
2.5.13 OA SET DAC CHANNEL VALUE..... 31
2.5.14 OA APPLY DAC VALUE TO PHYSICAL OUTPUT..... 32
2.5.15 OA SET CHANNEL AS RTD..... 33
2.5.16 OA GET RTD CHANNEL VALUE (Ohm)..... 34
2.5.17 OA SET RTD UPDATE TIME..... 35
2.5.18 OA SET CHANNEL AS DIGITAL INPUT..... 36
2.5.19 OA GET DIGITAL INPUT STATUS..... 37
2.5.20 OA SET CHANNEL AS HIGH IMPEDANCE..... 38
2.5.21 OA SET PWM..... 39
2.5.22 OA SET LED..... 40
2.5.23 GET FW VERSION..... 41
2.5.24 REBOOT EXPANSION..... 42
2.5.25 SAVE INFORMATION IN FLASH MEMORY..... 43
2.5.26 READ INFORMATION FROM FLASH..... 44
2.5.27 OA SET TIMEOUT TIME TO FORCE DEFAULT VALUES..... 45
2.5.28 OA SET DAC CHANNEL VALUE AFTER COMMUNICATION TIMEOUT..... 46
2.5.29 OA SET PWM CHANNEL VALUE AFTER COMMUNICATION TIMEOUT..... 47
2.5.30 OA GET CHANNEL CONFIGURATION..... 48
3 ADD NEW OPTA EXPANSION..... 49
3.1 Introduction..... 49
3.2 How to make new FW..... 49
3.2.1 OptaNewExpansion class..... 50
3.3 Overriding Basic Virtual Methods..... 51
3.3.1 Performance request..... 52
3.4 OptaNewExpansionCfg.h..... 53
3.5 Sharing information between FW and the controller..... 53
3.6 The NewExpansion Application Class..... 54
3.6.1 About the execute() function..... 55
3.6.2 Mandatory functions to be implemented in NewExpansionExpansion class..... 56
3.7 Controller additional initialization function for Custom expansion..... 57
3.8 NewExpansion Enumerative Type..... 58
3.9 About I2C Messages Definition..... 59
4 CUSTOM EXPANSION TYPE..... 61
4.1 Expansion Type..... 61
4.2 Register Expansion..... 61
4.3 How To Retrieve Register Expansion Type..... 61

4.4 Different Overloads..... 62

1 INTRODUCTION

This library (Arduino OptaBluePrint) is intended to be used on **Arduino OPTA** (use Controller class) and **Arduino OPTA Digital/Analog** (use Module class) expansion.

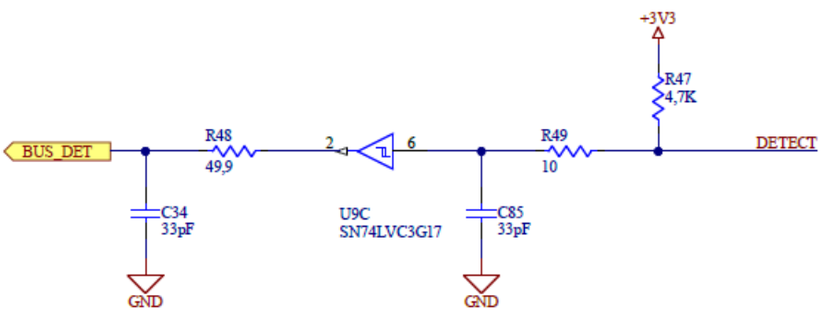
A single Arduino OPTA can control several **Arduino OPTA Expansions**. More different types of expansion are expected, and this library is ready to add them.

1.1 Hardware

Arduino OPTA and Arduino OPTA Expansions share an I2C bus and a sort of UART bus (OPTA Controller TX is wired on all OPTA Expansion RX).

Arduino **OPTA Controller** has a pin that goes *toward* OPTA expansion:

- **DETECT** pin. Please note that this pin can only be an input due to hardware implementation.



Each **OPTA expansion** has 2 pins connected on the expansion bus:

- **DETECT IN** goes toward the OPTA controller (Arduino OPTA)
- **DETECT OUT** goes toward another Arduino OPTA Expansion possibly connected

The Serial bus is used only to upgrade the Firmware (FW) of the expansions (they do not have a USB connector mounted). Basically, the controller sends an I2C message to expansion asking to go in bootloader mode. Once the bootloader is active, the Serial Line is used to download the new FW into the expansion. Then the controller tells the bootloader to reset the board and the upgrade is done.

1.1.1 OPTA AUX Port

The AUX port has 10 Pins, Pin 1 (3.3V) and Pin 8 (5V) are not used to power expansion modules. The part number of the board-to-board connector is **KYOCERA AVX 009159010061916**

3.3V	1	6	SDA
	2	7	SCL
RX	3	8	5V
TX	4	9	GND
DET	5	10	GND

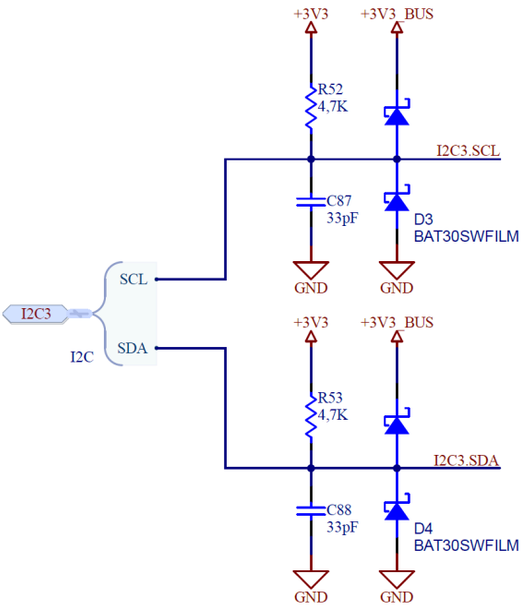




1.1.2 I2C bus

Arduino OPTA is always the master of the I2C, while the other expansions as **Arduino OPTA Expansion** are always slave on the I2C.

I2C Pins on Arduino OPTA Controller (pullup 4,7K ohms resistors are installed).



1.1.3 Switching Power Supply

There are two switching power supply available from Finder: 12W (2A/24V) for 1 controller + 5 expansions) and 25W for 2 groups of 1 controller + 5 expansions (3A/24V).

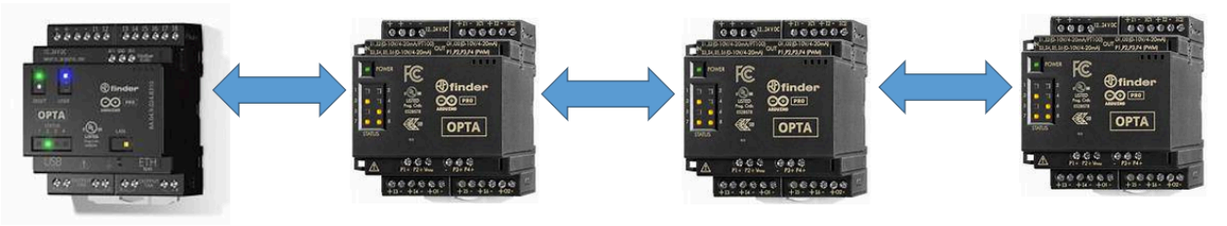


State	Green LED
OK	
Short Circuit	
Thermal Limit	OFF

1.2 Assign Addresses Process

1.2.1 Purposes of the Assign Addresses Process

Controller (Arduino OPTA) will be always placed in the leftmost position. One or more expansions (not necessarily of the same type) can be chained on the right of the controller.



OPTA CONTROLLER

OPTA EXPANSION #0
@=0x0A
@=0x12
@=0x0B

OPTA EXPANSION #1
@=0x0A
@=0x11
@=0x0C

OPTA EXPANSION #2
@=0x0A
@=0x10
@=0x0D

At the end of the **Assign Addresses Process**:

1. An expansion must have always the same address (this is because the controller sketch must continue to address the same device, if addresses were chosen randomly the controller logic will have to change depending on the assigned addresses)
2. The expansion closest to Controller must have address 0x0B, other expansions must have addresses that are increase of 1 going in the right direction. So, in our case shown above:
    - Expansion 0 gets address 0x0B
    - Expansion 1 gets address 0x0C
    - Expansion 2 gets address 0x0D
3. This process must not be dependent on the order the devices are powered up (or reset, if you will). As soon as 1 device is wired and powered up all the **Assign Addresses Process** is refreshed.

1.2.2 PIN Usage

Controller **DETECT** pin is always an INPUT (pullup): if HIGH there is no need to assign addresses, if LOW there are Expansions without an assigned address (pin is connected to the GND in this case). At the reset the controller always checks the DETECT pin. If it is LOW, the Controller enters the Assign Addresses Mode. The controller remains in this mode until the DETECT pin goes HIGH.

At the reset, each expansion calls the ***reset()*** function (called by ***begin()***). The ***reset***() function performs the following operation:

- stops and de-initialize the I2C (this guarantees that expansion does not receive any message until the I2C is properly initialized again)
- puts the DETECT IN pin to LOW (this signals the Expansion has not valid address)
- puts the DETECT OUT pin LOW for **OPTA_MODULE_DETECT_OUT_LOW_TIME:1000** ms (to notify other expansions that one expansion on the left has been reset)
- puts the DETECT OUT pin as INPUT_PULLUP
- remain in this state until the DETECT OUT becomes HIGH
- As soon as the DETECT OUT becomes HIGH, the I2C is initialized as slave with the default address (**OPTA_DEFAULT_SLAVE_I2C_ADDRESS:0x0A**)

After reset (see ***updatePinStatus()***) the pin usage on Expansion depends upon the address acquisition.

If address has NOT been acquired:

- the **DETECT IN** (toward Controller) is kept LOW.

If address has been acquired:

- the **DETECT IN** is briefly configured as INPUT to check if the **DETECT OUT** of the expansion on left is LOW (the expansion on the left is resetting its own address)
- the **DETECT OUT** is checked, if it becomes LOW this means that an expansion has been inserted on the right

The previous 2 conditions and the reception of the RESET message from the controller trigger again the function ***reset()*** so that the complete **Assign Address Process** is performed when one of these events is detected by one of the Expansion that already has an address.

1.2.3 RGB LED Usage

On Arduino Opta expansions the RGB LED is used to signal the status of the expansion regarding the Address assignment:

- **Blue**  Expansion has no address and is waiting for other expansion to get their own address; DETECT IN is HIGH, DETECT OUT is LOW
- **Red**  Expansion is ready to get the I2C address; DETECT IN is HIGH, DETECT OUT is HIGH
- **Green**  I2C address is acquired; DETECT IN is LOW, DETECT OUT is LOW or HIGH

1.2.4 I2C Assign Address Process

At start-up the controller sends to address **OPTA_CONTROLLER_FIRST_AVAILABLE_ADDRESS:0x0B** (this is guaranteed to be always the first device on the right of the controller when addresses have been assigned) the RESET message: this is done to deal with the case that expansions have already addresses assigned and the controller is somehow reset or switched off and on again. This message will trigger the first expansion to call the ***reset()*** function (see above).

After **OPTA_CONTROLLER_SETUP_INIT_DELAY:2000** msec, if the **DETECT** pin is LOW the Controller starts the first phase of the assign addresses procedure.

During the phase 1, the controller sends:

- A ***'SET ADDRESS MESSAGE'*** to I2C default address **OPTA_DEFAULT_SLAVE_I2C_ADDRESS:0x0A**; the new address is set to **OPTA_CONTROLLER_FIRST_TEMPORARY_ADDRESS:0x10 = OPTA_CONTROLLER_FIRST_AVAILABLE_ADDRESS:0x0B** + **OPTA_CONTROLLER_MAX_EXPANSION_NUM:5** (0x0B + 5 = 0x10) at first and then increased one after each expansion has successfully set the address. All expansion, due to the "reset" procedure, have the I2C in a de-initialized state, except the rightmost one (because its **DETECT OUT** is HIGH): for this reason, only the rightmost expansion receives the message and configures its address. Once the Expansion gets the address

it put its **DETECT IN** pin HIGH to leave the other expansions on the left the chance to get their own address.
- A ***'GET ADDRESS AND TYPE'*** message is sent to the last address set: this is done to ensure that the expansion has correctly obtained the address. With this message also a code that represents which expansion is attached is sent to the controller so that it can know what expansion types are dealing with (this makes sense only if other expansion types than OPTA DIGITAL/ANALOG will be available - but this is expected). If an answer is received the address to be sent is increased by one and the controller starts over: does the DETECT pin is LOW ?
    a. If **DETECT** is LOW: go to point 1
    b. Otherwise, continue with normal operations.

At the end of this phase:

- All expansions have a temporary address [from 0x10 on last end expansion (on the right) to 0x15 on the left if expansion exists]
- The controller knows the number of expansions and their types (and their temporary addresses)

However, the addresses are not always the same: the first device on the right of the controller will get an address that depends on the number of expansions chained together.

The Controller detects the end of this first phase by the fact that the **DETECT** pin has a transition LOW -> HIGH. When the Controller detects that state, it starts the second phase. Starting from the device with the higher address (which is the one on right of the controller), the controller reassigns the addresses starting from **OPTA_CONTROLLER_FIRST_AVAILABLE_ADDRESS:0x0B**. At the end of this phase all the expansions have always the same address that only depends upon the position in the expansion chain: from left to right 0x0B,0x0C, 0x0D, 0x0E and 0x0F.

During this second phase (phase 2) the Controller also creates the data structures used to deal with all the expansions. In particular:

- num_of_exp --> number of expansions detected: **OPTA_CONTROLLER_MAX_EXPANSION_NUM:5** maximum)
- exp_add[] --> is an array that holds the addresses assigned to each expansion (index 0 is the leftmost expansion index [num_of_exp - 1] is the rightmost expansion)
- exp_type[] --> is an array that holds the type of each expansion (index 0 is the leftmost expansion index [num_of_exp - 1] is the rightmost expansion)
- exp_index[] --> is an array that holds the index in array of data structures that holds all the information related to a certain type of expansion. To explain this let's have for example the following situation:
    1. expansion type A (index 0)
    2. expansion type B (index 1)
    3. expansion type A (index 2)
    4. expansion type B (index 3)

So, we have 2 expansions of type A and 2 expansions of type B. The controller then will create 2 arrays of different structures (classes) to hold the information of 2 different types of expansion. For example, Digital expansions are defined and the correspondent array is kept in the variables:

- uint8_t opta_digital_num;

- OptaDigitalData *opta_digitals;

Suppose that OPTA digital is type A and that there is a different kind of expansion (maybe an OPTA ANALOG) which is type B. Then we will have:

- opta_digital_num = 2.
- opta_analog_num = 2.
- opta_digitals is an array with 2 positions
- opta_analogs is an array with 2 positions

in this situation:

- exp_index[0] is 0 to indicate that (being the device at index 0 of type A or DIGITAL, this is got from exp_type[0]) all the information related to this device will be hold the opta_digitals[0] array position.
- exp_index[1] is 0 to indicate that (being the device at index 1 of type B or ANALOG, this is got from exp_type[1]) all the information related to this device will be hold the opta_analogs[0] array position.
- exp_index[2] is 1 to indicate that (being the device at index 2 of type A or DIGITAL, this is got from exp_type[2]) all the information related to this device will be hold the opta_digitals[1] array position.
- exp_index[3] is 1 to indicate that (being the device at index 3 of type B or ANALOG, this is got from exp_type[3]) all the information related to this device will be hold the opta_analogs[1] array position.

Using this structure and given an index (the index is always the physical position of the expansions in the chain from left to right starting from 0) the controller knows always which expansion is dealing with and how to retrieve / write information to it.

Say we want to know the status of a pin on expansion which has index 2. The controller goes in exp_type[2] and gets the type of the device (type A or DIGITAL in our example), then it goes to exp_index[2] and retrieves the position 1. With this it will retrieve information from opta_digitals[1].

1.2.5 I2C Addresses for 1 to 5 Expansions

Addresses (in hexadecimal, in decimal) used by controller/expansions (up to 5 expansions):

	init	Phase 1	Phase 2
1	0x0A	0x10	0x0B
2	0x0A	0x11	0x0B
	0x0A	0x10	0x0C
3	0x0A	0x12	0x0B
	0x0A	0x11	0x0C
	0x0A	0x10	0x0D
4	0x0A	0x13	0x0B
	0x0A	0x12	0x0C
	0x0A	0x11	0x0D
	0x0A	0x10	0x0E
5	0x0A	0x14	0x0B
	0x0A	0x13	0x0C
	0x0A	0x12	0x0D
	0x0A	0x11	0x0E
	0x0A	0x10	0x0F

init	Phase 1	Phase 2	
10	16	11	1
10	17	11	2
10	16	12	
10	18	11	3
10	17	12	
10	16	13	
10	19	11	4
10	18	12	
10	17	13	
10	16	14	
10	20	11	5
10	19	12	
10	18	13	
10	17	14	
10	16	15	

1.2.6 OptaModuleProtocol.cpp

1.2.6.1 Begin()

```
/* ----------------------------------------------------------------------------
 */
/* Module begin */
/* ----------------------------------------------------------------------------
 */
void Module::begin() {

  initStatusLED();
#ifdef DEBUG_SERIAL
  Serial.begin(115200);
  delay(5000);
  // while(!Serial) { }
  Serial.println("[Log]: START");
#endif


  reset();

  /* Set up callbacks */
  if (Module::expWire != nullptr) {
    Module::expWire->onReceive(receive_event);
    Module::expWire->onRequest(request_event);
  }
}
```

1.2.6.2 Reset()

```
/* ------------------------------------------------------------------------ */
/* Handle reset                                                             */
/* ------------------------------------------------------------------------ */
void Module::reset() {

  if (Module::expWire != nullptr) {
    Module::expWire->end();
  }
  #ifdef USE_CONFIRM_RX_MESSAGE
  confirm_address_reception = false;
  #endif

  setStatusLedWaitingForAddress();
  /* put address to invalid */
  wire_i2c_address = OPTA_DEFAULT_SLAVE_I2C_ADDRESS;
  rx_i2c_address = OPTA_DEFAULT_SLAVE_I2C_ADDRESS;

  /* detect_in (toward Controller) as Output */
  pinMode(detect_in, OUTPUT);
  /* put detect in pin to LOW -> signal the MODULE wants an address */
  digitalWrite(detect_in, LOW);

  /* detect_out (toward other Module) as output*/
  pinMode(detect_out, OUTPUT);
  digitalWrite(detect_out, LOW);
  delay(OPTA_MODULE_DETECT_OUT_LOW_TIME);

  pinMode(detect_out, INPUT_PULLUP);

  while (is_detect_out_low()) {

  }

  /* put I2C address to the default one */
  if (Module::expWire != nullptr) {
    Module::expWire->begin(OPTA_DEFAULT_SLAVE_I2C_ADDRESS);
  }

}
```

1.2.6.3 Is_detect_out_low()

```
/* ------------------------------------------------------------------------ */
bool Module::is_detect_out_low() {
  /* ------------------------------------------------------------------------
   */

  pinMode(detect_out, INPUT_PULLUP);
  delay(1);
  PinStatus dtcout = digitalRead(detect_out);

  if (dtcout == LOW) {
    int num = 0;
    while ( dtcout == LOW && num < OPTA_MODULE_DEBOUNCE_NUMBER_OUT) {
      dtcout = digitalRead(detect_out);
      if (dtcout == HIGH) {
        break;
      }
      delay(OPTA_MODULE_DEBOUNCE_TIME_OUT);
      num++;
    }
  }
  return (dtcout == LOW);
}
```

1.2.6.4 updatePinStatus()

```
/* ---------------------------------------------------------------------------- */
void Module::updatePinStatus() {
/* ---------------------------------------------------------------------------- */

#if defined DEBUG_SERIAL && defined DEBUG_UPDATE_PIN_ENABLE
  Serial.print("- UPDATE PIN ");
#endif

  if (!addressAcquired()) {
#if defined DEBUG_SERIAL && defined DEBUG_UPDATE_PIN_ENABLE
    Serial.println("ADDRESS not ACQUIRED");
#endif

    pinMode(detect_out, INPUT_PULLUP);
    if(digitalRead(detect_out) == LOW) {
      reset();
    }

    setStatusLedReadyForAddress();
    /* detect_in (toward Controller) as Output */
    pinMode(detect_in, OUTPUT);
    /* put detect in pin to LOW -> signal the MODULE wants an address */
    digitalWrite(detect_in, LOW);
  } else {
#if defined DEBUG_SERIAL && defined DEBUG_UPDATE_PIN_ENABLE
    Serial.print("ADDRESS ACQUIRED ");
    Serial.println(wire_i2c_address, HEX);
#endif
    setStatusLedHasAddress();
    #ifdef USE_CONFIRM_RX_MESSAGE
    if(confirm_address_reception) {
    #endif
      if (is_detect_in_low()) {
        reset_required = true;
      } else if (is_detect_out_low()) {
        reset_required = true;
      } else {
        pinMode(detect_out, INPUT_PULLUP);
        digitalWrite(detect_in, HIGH);
      }
    #ifdef USE_CONFIRM_RX_MESSAGE
    }
    #endif
  }
}
```

1.3 About Classes

Module is the base class for Expansions.

This class handles the assign addresses process and so it parses the related messages and handles the related Pin setting.

There are mainly 4 public functions in this class:

1. begin() -> initialize HW stuff
2. parse_RX() -> parse the messages related to assign addresses process
3. update() -> just call updatePinStatus() which is a private function handling the pin setting related to assign addresses process
4. end() -> does nothing

OptaDigital is the specific class for Arduino Opta Digital management. And OptaAnalog is the specific class for Arduino Opta Analog management.

So, it handles all the hardware stuff specific for that hardware: in particular, analog conversion, digital input and digital output.

These classes override the functions of the Module class; however, the base class functions are called into the derived class versions.

IMPORTANT TO REMEMBER:

When new hardware is added: ALWAYS CALL the base version function of the Module class, since these functions deal with the stuff that all expansions are supposed to handle (such the assign addresses process). It is supposed that a base Module class will never be instantiated.

Page intentionally left blank

2 OptaBluePrint PROTOCOL

This CHAPTER describes the OptaBluePrint protocol used on I2C bus. In this text we won't use any actual value, but the ‘#define’ used through the code. The actual value can be found in the Protocol.h file.

2.1 Header

Each message has a header with 3 bytes:

- The first byte is the COMMAND (what to do)
- The second byte is the ARGUMENT (the target of the COMMAND)
- The third byte is the LENGTH (this specifies how many bytes follow the HEADER i.e. the payload length). Please note:
    - if CRC is used, the LENGTH does not include the last CRC8 byte
    - LENGTH can be 0 if not payload is expected

2.1.1 Commands

The first byte of each message is the COMMAND code.

2.1.1.1 Controller commands

These are the commands (first byte of the message) sent by the Controller and received by the Expansions:

- set (used to set something in the expansion), value BP_CMD_SET
- get (used to get something in the expansion), value BP_CMD_GET

2.1.1.2 Expansions commands (answers)

These are the commands (first byte of the message) sent by the Expansions and received by the Controller:

- set answer (used to set something in the expansion), value BP_ANS_SET
- get answer (used to get something in the expansion), value BP_ANS_GET

2.1.2 Arguments

The second byte of the message is the ARGUMENT: this byte is intended to specify which is the target or the action performed by the COMMAND.

2.1.2.1 Example of arguments

- ARG_CONTROLLER_RESET (value 0x01) specify that the SET COMMAND is aimed to "reset" the Expansion to its default I2C address.
- ARG_ADDRESS (value 0x02) specify that the SET COMMAND is aimed to set the I2C address of the EXPANSION.

2.2 Payload

It follows the HEADER and must have LENGTH bytes; it specifies the value of the argument.

The payload specifies the value of the argument: for example, ARG_ADDRESS in SET COMMAND is aimed to set the address of the I2C expansion, therefore the PAYLOAD must be the value of the address. In this case the LENGTH field must be set to 1 specifying that the PAYLOAD length is 1 byte long. Different messages have different payloads, with different lengths.

2.3 CRC

By default, the protocol adds a CRC8 at the end of each message immediately after the payload, the use of the CRC can be disable removing the definition of BP_USE_CRC. When used the message is discarded and not used if the CRC is wrong. This CRC is based on an 8-bit CRC using the following polynomial:

P(x) = x^8 + x^2 + x^1 + 1

All bytes of the message from COMMAND to the last bytes of payload are included in the CRC calculation.

2.4 Messages Unhandled

If an expansion receives a message which is not supposed to handle (and this message is expecting an answer from the expansion) the first 2 bytes of the answer will be 0xFA, 0xFE: this sequence signals that the expansion received the request, but this request is not supported by this expansion.

2.5 Messages

The following describes each message and the related answer:

2.5.1 RESET EXPANSION I2C ADDRESS

Apply to: All expansion types

When an expansion receives this message, it resets its own I2C address to the default one (0x0A) and de-initialize I2C bus (the I2C bus is automatically initialized if the DETECT IN, DETECT OUT pins are in the correct state)

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_CONTROLLER_RESET (0x01)
    - LEN_CONTROLLER_RESET (0x01)
  - Payload:
    - CONTROLLER_RESET_CODE (0x56)
  - CRC
- Expansion answer
  - No answer expected

2.5.2 SET I2C ADDRESS

Apply to: All expansion types

When an expansion receives this message, it sets its own I2C address to the one specified in the payload.

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_ADDRESS (0x02)
    - LEN_ADDRESS (0x01)
  - Payload:
    - the new I2C address to be used (1 byte)
  - CRC

- Expansion answer
  - No answer expected

2.5.3 GET I2C ADDRESS AND TYPE

Apply to: All expansion types

When an expansion receives this message, it answers with its own address and type.

- Controller request
  - Header:
    - BP_CMD_GET (0x02)
    - ARG_ADDRESS_AND_TYPE (0x03)
    - LEN_ADDRESS_AND_TYPE (0x00)
  - Payload: None
  - CRC

- Expansion answer
  - Header:
    - BP_ANS_GET (0x03)
    - ANS_ARG_ADDRESS_AND_TYPE (0x03)
    - ANS_LEN_ADDRESS_AND_TYPE (0x02)
  - Payload:
    - the address (1 byte)
    - the type of expansion (1 byte)
  - CRC

Type of expansion values:

- Opta Digital Invalid -> 0x01
- Opta Digital Mechanical -> 0x02,
- Opta Digital State Solid -> 0x03,
- Opta Analog -> 0x04,
- Opta Custom -> 0x05

Custom expansion gets a dynamic expansion type See chapter About Custom Expansion Type

2.5.4 OD GET DIGITAL VALUES

Apply to: Opta Digital

When an expansion receives this message, it answers with the status of digital inputs.

- Controller request
  - Header:
    - BP_CMD_GET (0x02)
    - ARG_OD_GET_DIGITAL_INPUTS (0x04)
    - LEN_OD_GET_DIGITAL_INPUTS (0x00)
  - Payload: None
  - CRC

- Expansion answer
  - Header:
    - BP_ANS_GET (0x03)
    - ANS_ARG_ADDRESS_AND_TYPE (0x04)
    - ANS_LEN_OD_GET_DIGITAL_INPUTS (0x02)
  - Payload:
    - the status of digital inputs (2 byte)
  - CRC

Note: the status of digital inputs is a bit mask word (2 bytes): each bit of the word represents the status of digital input (1 means HIGH, 0 means LOW).

2.5.5 OD GET ANALOG VALUES

Apply to: Opta Digital

When an expansion receives this message, it answers with the value of the analog pin received as argument (the analog value is the raw analog value in bits).

- Controller request
  - Header:
    - BP_CMD_GET (0x02)
    - ARG_OD_GET_ANALOG_INPUT (0x05)
    - LEN_OD_GET_ANALOG_INPUT (0x01)
  - Payload:
    - the number of the PIN (1 byte)
  - CRC

- Expansion answer
  - Header:
    - BP_ANS_GET (0x03)
    - ANS_ARG_OD_GET_ANALOG_INPUT (0x05)
    - ANS_LEN_OD_GET_ANALOG_INPUT (0x02)
  - Payload:
    - the analog value of the PIN (2 byte) - LSB first
  - CRC

Note: the analog value is a WORD (2 bytes) - first is sent the LSB then the MSB.

2.5.6 OD GET ANALOG ALL VALUES

Apply to: Opta Digital

When an expansion receives this message, it answers with the analog values of all its PINs.

- Controller request
  - Header:
    - BP_CMD_GET (0x02)
    - ARG_OD_GET_ALL_ANALOG_INPUTS (0x07)
    - LEN_OD_GET_ALL_ANALOG_INPUTS (0x00)
  - Payload: None
  - CRC

- Expansion answer
  - Header:
    - BP_ANS_GET (0x03)
    - ANS_ARG_OD_GET_ALL_ANALOG_INPUTS (0x07)
    - ANS_LEN_OD_GET_ALL_ANALOG_INPUTS (32 bytes)
  - Payload:
    - the analog value of all the 16 PINs (each pin is 2 bytes) - LSB first
  - CRC

Note: the payload is array of 16 words (LSB first) each value represents the analog value of the corresponding PIN.

2.5.7 OD SET DIGITAL OUTPUTS

Apply to: Opta Digital

When an expansion receives this message, it sets the digital outputs

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OD_SET_DIGITAL_OUTPUTS (0x06)
    - LEN_OD_SET_DIGITAL_OUTPUTS (0x01)
  - Payload:
    - The status of all digital outputs (1 byte)
  - CRC

- Expansion answer
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OD_SET_DIGITAL_OUTPUTS (0x06)
    - ANS_LEN_OD_SET_DIGITAL_OUTPUTS (0x00)
  - Payload: None
  - CRC

Note (1): the status of all digital output is a bitmask: each bit corresponds to the status the PIN will be set upon by the expansion at the reception of the SET message (1 --> HIGH, 0 --> LOW)

Note (2): the Expansion answer is requested in order to ensure that the message is properly received (CRC is OK), if the message answer is not received the Controller will trigger the related callback function (if set).

2.5.8 OD SET DEFAULT OUTPUT VALUES AND TIMEOUT

Apply to: Opta Digital

Each Digital expansion monitors the I2C and expects to receive an I2C message within a certain timeout, if the timeout expires the Expansion resets its output to the default value. This is a safety policy meant to avoid the fact that output sticks to a value in case the controller for some reason cannot communicate with expansion. This message sets the default value for the Opta Digital expansion and the related timeout. A timeout equal to 0xFFFF means no timeout.

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OD_DEFAULT_AND_TIMEOUT (0x08)
    - LEN_OD_DEFAULT_AND_TIMEOUT (0x03)
  - Payload:
    - the status of all digital outputs (1 byte)
    - the timeout (2 bytes) - LSB first
  - CRC

- Expansion answer
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OD_DEFAULT_AND_TIMEOUT (0x08)
    - ANS_LEN_OD_DEFAULT_AND_TIMEOUT (0x00)
  - Payload: None
  - CRC

Note (1): the status of all digital output is a bitmask: each bit corresponds to the status the PIN will be set upon by the expansion at the reception of the SET message (1 --> HIGH, 0 --> LOW)

Note (2): the timeout is a word (2 bytes) LSB sent first

Note (3): the Expansion answer is requested in order to ensure that the message is properly received (CRC is OK), if the message answer is not received the Controller will trigger the related callback function (if set).

2.5.9 OA SET CHANNEL AS ADC

Apply to: Opta Analog

Configure an Opta Analog channel as ADC

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OA_CH_ADC (0x09)
    - LEN_OA_CH_ADC (0x07)
  - Payload:
    - channel (1 byte) from 0 to 7
    - ADC type (1 byte) (0 voltage adc, 1 current adc)
    - use pulldown (1 Enable, 2 Disable)
    - use rejection (1 Enable, 2 Disable)
    - use diagnostic (1 Enable, 2 Disable)
    - number of points of the moving average (max 255)
    - add ADC to another function (1 Enable, 2 Disable)
  - CRC

- Expansion answer (ACK answer):
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OA_ACK (0x20)
    - ANS_LEN_OA_ACK (0)
  - Payload: no payload
  - CRC

2.5.10 OA GET ADC VALUE

Apply to: Opta Analog

Get the current value of ADC channel (0x0000 to 0xFFFF)

- Controller request
  - Header:
    - BP_CMD_GET (0x02)
    - ARG_OA_GET_ADC (0x0A)
    - LEN_OA_GET_ADC (0x01)
  - Payload:
    - channel (1 byte) from 0 to 7
  - CRC

- Expansion answer:
  - Header:
    - BP_ANS_GET (0x03)
    - ANS_ARG_OA_GET_ADC (0x0A)
    - ANS_LEN_OA_GET_ADC (0x03)
  - Payload:
    - channel (1 byte) from 0 to 7
    - the analog value of the channel (2 byte) - LSB first
  - CRC

2.5.11 OA GET ALL ADC VALUE AT ONCE

Apply to: Opta Analog

Get the current value of ADC channel (0x0000 to 0xFFFF)

- Controller request
  - Header:
    - BP_CMD_GET (0x02)
    - ARG_OA_GET_ALL_ADC (0x0B)
    - LEN_OA_GET_ALL_ADC (0x00)
  - Payload: None
  - CRC

- Expansion answer:
  - Header:
    - BP_ANS_GET (0x03)
    - ANS_ARG_OA_GET_ALL_ADC (0x0B)
    - ANS_LEN_OA_GET_ALL_ADC (0x10)
  - Payload:
    - 16 bytes (2 for each channel) - LSB first
  - CRC

2.5.12 OA SET CHANNEL AS DAC

Apply to: Opta Analog

Configure an Opta Analog channel as DAC

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OA_CH_DAC (0x0C)
    - LEN_OA_CH_DAC (0x05)
  - Payload:
    - channel (1 byte) from 0 to 7
    - DAC type (1 byte) (0 voltage dac, 1 current dac)
    - limit current (1 Enable, 2 Disable)
    - use slew rate (1 Enable, 2 Disable)
    - slew rate (0 - 16)
  - CRC

- Expansion answer (ACK answer):
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OA_ACK (0x20)
    - ANS_LEN_OA_ACK (0)
  - Payload: no payload
  - CRC

2.5.13 OA SET DAC CHANNEL VALUE

Apply to: Opta Analog

Set Opta Analog DAC channel value (0x0000 to 0x1FFF)

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OA_SET_DAC (0x0D)
    - LEN_OA_SET_DAC (0x04)
  - Payload:
    - channel (1 byte) from 0 to 7
    - DAC value (2 bytes) (0 - 8191) - LSB first
    - UPDATE DAC immediately (1 - immediately, 0 - no immediate update)
  - CRC

- Expansion answer (ACK answer):
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OA_ACK (0x20)
    - ANS_LEN_OA_ACK (0)
  - Payload: no payload
  - CRC

2.5.14 OA APPLY DAC VALUE TO PHYSICAL OUTPUT

Apply to: Opta Analog

If set DAC message above is used with UPDATE DAC immediately equal to 0 the DAC value is not output until this message is received (this can be used to update DAC synchronously)

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OA_SET_ALL_DAC (0x22)
    - LEN_OA_SET_ALL_DAC (0x00)
  - Payload: no payload
  - CRC

- Expansion answer (ACK answer):
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OA_ACK (0x20)
    - ANS_LEN_OA_ACK (0)
  - Payload: no payload
  - CRC

2.5.15 OA SET CHANNEL AS RTD

Apply to: Opta Analog

Configure an Opta Analog channel as RTD (IEEE Floating format)

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OA_CH_RTD (0x0E)
    - LEN_OA_CH_RTD (0x06)
  - Payload:
    - channel (1 byte) from 0 to 7
    - use 3 wire (1 byte) (1 Enable, 2 Disable)
    - current mA as float on 4 bytes
  - CRC

- Expansion answer (ACK answer):
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OA_ACK (0x20)
    - ANS_LEN_OA_ACK (0)
  - Payload: no payload
  - CRC

2.5.16 OA GET RTD CHANNEL VALUE (Ohm)

Apply to: Opta Analog

Get Opta Analog RTD channel value (IEEE float)

- Controller request
  - Header:
    - BP_CMD_GET (0x02)
    - ARG_OA_GET_RTD (0x0F)
    - LEN_OA_GET_RTD (0x01)
  - Payload:
    - Channel (1 byte) from 0 to 7
  - CRC

- Expansion answer:
  - Header:
    - BP_ANS_GET (0x03)
    - ANS_ARG_OA_GET_RTD (0x0F)
    - ANS_LEN_OA_GET_RTD (0x05)
  - Payload:
    - channel (1 byte) from 0 to 7
    - value 4 bytes as float
  - CRC

2.5.17 OA SET RTD UPDATE TIME

Apply to: Opta Analog

Configure the update time of RDT channel (this is used for each RTD channel), Take into account that 3 wire RTD takes around 800 ms per channel.

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OA_SET_RTD_UPDATE_TIME (0x10)
    - LEN_OA_SET_RTD_UPDATE_TIME (0x02)
  - Payload:
    - update time (ms) - 2 bytes - LSB first
  - CRC

- Expansion answer (ACK answer):
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OA_ACK (0x20)
    - ANS_LEN_OA_ACK (0)
  - Payload: no payload
  - CRC

2.5.18 OA SET CHANNEL AS DIGITAL INPUT

Apply to: Opta Analog

Configure an Opta Analog channel as Digital Input

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OA_CH_DI (0x11)
    - LEN_OA_CH_DI (0x09)
  - Payload:
    - channel (1 byte) from 0 to 7
    - filter comparator (1 Enable, 2 Disable)
    - invert comparator output (1 Enable, 2 Disable)
    - enable comparator (1 Enable, 2 Disable)
    - use simple debounce (1 Enable, 2 Disable) - disable means Integrator debounce
    - scale voltage comparator (1 Enable, 2 Disable)
    - threshold voltage comparator (0 - 31)
    - current sink (0 - 31)
    - debounce time (0 - 31)
  - CRC

- Expansion answer (ACK answer):
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OA_ACK (0x20)
    - ANS_LEN_OA_ACK (0)
  - Payload: no payload
  - CRC

2.5.19 OA GET DIGITAL INPUT STATUS

Apply to: Opta Analog

Get Opta Analog DIGITAL INPUT status

- Controller request
  - Header:
    - BP_CMD_GET (0x02)
    - ARG_OA_GET_DI (0x12)
    - LEN_OA_GET_DI (0x00)
  - Payload: None
  - CRC

- Expansion answer:
  - Header:
    - BP_ANS_GET (0x03)
    - ANS_ARG_OA_GET_DI (0x12)
    - ANS_LEN_OA_GET_DI (0x01)
  - Payload:
    - all DI status (1 byte) as bit mask
  - CRC

2.5.20 OA SET CHANNEL AS HIGH IMPEDANCE

Apply to: Opta Analog

Configure an Opta Analog channel as High Impedance

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OA_CH_DI (0x24)
    - LEN_OA_CH_DI (0x01)
  - Payload:
    - channel (1 byte) from 0 to 7
  - CRC

- Expansion answer (ACK answer):
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OA_ACK (0x20)
    - ANS_LEN_OA_ACK (0)
  - Payload: no payload
  - CRC

2.5.21 OA SET PWM

Apply to: Opta Analog

Set Opta digital PWM channel values

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OA_SET_PWM (0x13)
    - LEN_OA_SET_PWM (0x09)
  - Payload:
    - channel (1 byte) from 0 to 3
    - pwm period in usec (4 bytes) - LSB first
    - pwm pulse (high) in usec (4 bytes) - LSB first
  - CRC

- Expansion answer (ACK answer):
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OA_ACK (0x20)
    - ANS_LEN_OA_ACK (0)
  - Payload: no payload
  - CRC

2.5.22 OA SET LED

Apply to: Opta Analog

Set Opta digital PWM channel values

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OA_SET_LED (0x15)
    - LEN_OA_SET_LED (0x01)
  - Payload:
    - LEDs status (1 byte) as bitmask
  - CRC

- Expansion answer (ACK answer):
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OA_ACK (0x20)
    - ANS_LEN_OA_ACK (0)
  - Payload: no payload
  - CRC

2.5.23 GET FW VERSION

Apply to: All expansion types

Get FW version in the form MAJOR. MINOR. RELEASE.

- Controller request
  - Header:
    - BP_CMD_GET (0x02)
    - ARG_GET_VERSION (0x16)
    - LEN_GET_VERSION (0x00)
  - Payload: None
  - CRC

- Expansion answer:
  - Header:
    - BP_ANS_GET (0x03)
    - ANS_ARG_GET_VERSION (0x16)
    - ANS_LEN_GET_VERSION (0x03)
  - Payload:
    - major (1 byte)
    - minor (1 byte)
    - release (1 byte)
  - CRC

2.5.24 REBOOT EXPANSION

Apply to: All expansion types

Put expansion in Boot Loader Mode

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_REBOOT (0xF3)
    - LEN_REBOOT (0x02)
  - Payload:
    - first reboot command (1 byte) - value REBOOT_1_VALUE (0x58)
    - second reboot command (1 byte) - value REBOOT_2_VALUE (0x32)
  - CRC

- Expansion answer:
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_REBOOT (0xF3)
    - ANS_LEN_REBOOT (0x01)
  - Payload:
    - reboot confirmation code (1 byte) - value ANS_REBOOT_CODE (0x74)
  - CRC

2.5.25 SAVE INFORMATION IN FLASH MEMORY

Apply to: All expansion types

Save up to 32 bytes in the Data Flash of the device

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_SAVE_IN_DATA_FLASH (0x17)
    - LEN_SAVE_IN_DATA_FLASH (35)
  - Payload:
    - address 2 bytes - LSB first
    - size (1 - 32) 1 byte
    - 32 bytes of information (all 32 bytes must be provided but only 'size' are actually saved)
  - CRC

- Expansion answer: None

2.5.26 READ INFORMATION FROM FLASH

Apply to: All expansion types

Read up to 32 bytes from the data flash of the device

- Controller request
  - Header:
    - BP_CMD_GET (0x02)
    - ARG_GET_DATA_FROM_FLASH (0x18)
    - LEN_GET_DATA_FROM_FLASH (3)
  - Payload:
    - address 2 bytes - LSB first
    - size (1 - 32) 1 byte
  - CRC

- Expansion answer:
  - Header:
    - BP_ANS_SET (0x03)
    - ANS_ARG_GET_DATA_FROM_FLASH (0x19)
    - ANS_LEN_GET_DATA_FROM_FLASH (35)
  - Payload:
    - address 2 bytes - LSB first
    - size (1 - 32) 1 byte
    - 32 bytes of information (32 bytes are always read but information is only in the first 'size' of them)
  - CRC

2.5.27 OA SET TIMEOUT TIME TO FORCE DEFAULT VALUES

Apply to: Opta Analog

Set Opta Analog Timeout time, if the expansion does not receive messages from the controller for more than this time the outputs (DAC and PWM) are set to the values set by the messages ARG_OA_SET_DAC_DEFAULT or ARD_OA_SET_DEFAULT_PWM (see below for details).

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OA_SET_TIMEOUT_TIME (0x31)
    - LEN_OA_SET_RTD_UPDATE_TIME (0x02) // "shared define"
  - Payload:
    - timeout in ms (2 bytes) - LSB first
  - CRC

- Expansion answer (ACK answer):
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OA_ACK (0x20)
    - ANS_LEN_OA_ACK (0)
  - Payload: no payload
  - CRC

2.5.28 OA SET DAC CHANNEL VALUE AFTER COMMUNICATION TIMEOUT

Apply to: Opta Analog

Set Opta Analog DAC default channel value, this value will be applied on the channel when a timeout in the communication with the controller occurs, values from 0x0000 to 0x1FFF)

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARG_OA_SET_DAC_DEFAULT (0x3D)
    - LEN_OA_SET_DAC (0x04)
  - Payload:
    - channel (1 byte) from 0 to 7
    - DAC value (2 bytes) (0 - 8191) - LSB first
    - UPDATE DAC immediately (1 byte) - not used
  - CRC

- Expansion answer (ACK answer):
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OA_ACK (0x20)
    - ANS_LEN_OA_ACK (0)
  - Payload: no payload
  - CRC

2.5.29 OA SET PWM CHANNEL VALUE AFTER COMMUNICATION TIMEOUT

Apply to: Opta Analog

Set Opta digital PWM channel values

- Controller request
  - Header:
    - BP_CMD_SET (0x01)
    - ARD_OA_SET_DEFAULT_PWM (0x33)
    - LEN_OA_SET_PWM (0x09)
  - Payload:
    - channel (1 byte) from 0 to 3
    - pwm period in usec (4 bytes) - LSB first
    - pwm pulse (high) in usec (4 bytes) - LSB first
  - CRC

- Expansion answer (ACK answer):
  - Header:
    - BP_ANS_SET (0x04)
    - ANS_ARG_OA_ACK (0x20)
    - ANS_LEN_OA_ACK (0)
  - Payload: no payload
  - CRC

2.5.30 OA GET CHANNEL CONFIGURATION

Apply to: Opta Analog

Get the actual function of the channel (to be used to verify when the expansion has actually set up the requested function on that channel, in special cases when outputs are issue just once in a while and so it is recommended to check if the expansion has already set up the channel in advance)

- Controller request
  - Header:
    - BP_CMD_GET (0x02)
    - ARG_GET_CHANNEL_FUNCTION (0x40)
    - LEN_GET_CHANNEL_FUNCTION (0x01)
  - Payload:
    - channel (1 byte) from 0 to 7
  - CRC

- Expansion answer:
  - Header:
    - BP_ANS_GET (0x03)
    - ANS_GET_CHANNEL_FUNCTION (0x40)
    - LEN_ANS_GET_CHANNEL_FUNCTION (0x02)
  - Payload:
    - channel (1 byte) from 0 to 7
    - present channel function (1 byte)
      - HIGH_IMPEDANCE (0),
      - VOLTAGE_OUTPUT (1),
      - CURRENT_OUTPUT (2),
      - VOLTAGE_INPUT (3),
      - CURRENT_INPUT (4),
      - CURRENT_INPUT_LOOP_POWER (5),
      - RESISTANCE_MEASUREMENT (6),
      - DIGITAL_INPUT (7),
      - DIGITAL_INPUT_LOOP_POWER (8),
      - UNDEFINED (9),
      - RESISTANCE_MEASUREMENT_3_WIRES (10)
  - CRC

3 ADD NEW OPTA EXPANSION

3.1 Introduction

This chapter is intended for people who want to add support for a new expansion to OptaBluePrint.

To add new expansion to the OptaBluePrint library it is necessary to provide:

- a new FW (this is the 'sketch' running on the expansion hardware that receives command from Opta Controller and send back information)
- a new <Something>Expansion class which is derived from the Expansion class contained in this Library (in the following this be referred as the 'expansion class', this is intended to be all the necessary files to have a fully usable expansion on the Opta Controller)

The FW and expansion class must be developed in a different folder/repository, and no changes or PR should be required in this OptaBluePrint library. We suggest creating a separate library.

It is suggested that FW source files and the expansion class are put in the same folder for simplicity reasons (this allows us to share information between the FW and expansion class in an easy way).

3.2 How to make new FW

The FW will be composed by the following file

- An OptaSomething.h file
- An OptaSomething.cpp file
- An OptaSomethingCfg.h file (optional ?)
- A Something.ino sketch that will be the entry point to use implementation in the Opta<Something> files

Please note that the filename convention indicated above is just to keep consistent with the file naming used in OptaBluePrint library, but it is optional.

We recommend the sketch Something.ino to be placed in a different folder (something like 'firmware'): for example, the Digital expansion FW resides in folder ./firmware/Digital/Digital.ino but it uses the OptaDigital.h and .cpp in the ./src folder

So, a good starting point to develop an expansion FW is the following folder structure (suppose your expansion is called "NewExpansion")

/root_folder_of_new_expansion_library

|-firmware/NewExpansion/NewExpansion.ino

|-src/

|-OptaNewExpansion.h

|-OptaNewExpansion.cpp

|-OptaNewExpansionCfg.h (optional)

It is required that every FW version will be identified with a version number in this form: MAJOR.MINOR. RELEASE. MAJOR, MINOR and RELEASE are bytes (uint8_t); This defines are placed in OptaNewExpansionCfg.h

We recommend using an #ifdef NEWEXPANSION clause in the file OptaNewExpansion.cpp (or, alternatively, to use #ifndef ARDUINO_OPTA) so that your FW implementation file will not be compiled when the application sketch for Opta Controller is built.

NOTE:

In the following we suppose that the new expansion "name" is "NewExpansion"

3.2.1 OptaNewExpansion class

The files OptaNewExpansion file .h and .cpp must contain the implementation of the class that handles the Expansion FW.

This class must inherit from Module class. Module class declaration is contained in the OptaBlueModule.h header file so that this file must be included in OptaNewExpansion.h

The class will be something like

class OptaNewExpansion : public Module {

};

IMPORTANT:

The constructor for your new OptaNewExpansion class must always explicitly call the Module class constructor ***Module(TwoWire *tw, int _detect_in, int _detect_out)*** specifying:

- The TwoWire object used for I2C OptaBluePrint communication
- the DETECT IN pin used by the new expansion
- the DETECT OUT pin used by the new expansion

The Module class defines some pure virtual functions so that every expansion must implement these functions:

- ***virtual uint8_t getMajorFw();*** must return the Major FW version
- ***virtual uint8_t getMinorFw();*** must return the Minor FW version
- ***virtual uint8_t getReleaseFw();*** must return the Release FW version
- ***virtual std::string getProduct();*** must return a string containing a description of the new expansion. This description will be used to identify in a unique way the ***type*** of expansion. It must never happen that 2 different kinds of expansions have the same description. The number of bytes used is 32 (if the vector has more than 32 bytes, then only the first 32 are used). Product description must be chosen carefully in order to avoid the fact that different producers use the same description, so a combination of the producer plus a unique identifier for the product should be enough. For example, Opta Analog uses "Arduino Opta Analog" as product description (and only uses 21 byte).
- ***virtual void goInBootloaderMode();*** this function must implement a procedure to put the expansion in Bootloader mode in order to allow the upgrade of the FW. In case the Expansion does not provide the possibility to upgrade the FW just provide a dummy empty implementation.
- ***virtual void readFromFlash(uint16_t add, uint8_t *buffer, uint8_t dim);*** this function must implement the reading from flash (or anything similar) expansion device at address ***add*** of ***dim*** bytes, the bytes must be put into the ***buffer*** and the buffer must be

at least ***dim*** long. Pay attention: read from flash function is in any case "driven" by an I2C transaction from Opta Controller so the maximum ***dim*** is 32 bytes (i.e. FLASH or EEPROM or any form of implementation is read 32 bytes at each transaction). Also, the address is only 2 bytes wide because the I2C protocol reserves 2 bytes for the address. This is the default implementation you get for free by simply implementing this function (this means that at the controller level all is already implemented). Those limitations can be overcome by defining and handling a different I2C message. In case your expansion does not have this function just provide an empty, dummy implementation.

- ***virtual void writeInFlash(uint16_t add, uint8_t *buffer, uint8_t dim);*** this function must implement the writing into flash (or anything similar) expansion device at address ***add*** of ***dim*** bytes, the bytes copied from the ***buffer*** and so the buffer must be at least ***dim*** long. Pay attention: write into flash function is in any case "driven" by an I2C transaction from Opta Controller so the maximum writable ***dim*** is 32 bytes (i.e.FLASH or EEPROM or any form of implementation is written 32 bytes at each transaction). Also, the address is only 2 bytes wide because the I2C protocol reserves 2 bytes for the address. This is the default implementation you get for free by simply v this function (this means that at the controller level all is already implemented). That limitation can be overcome by defining and handling a different I2C message. In case your expansion does not have the possibility to write in flash just provide an empty, dummy implementation.
- ***virtual void initStatusLED();*** this function must provide a way to correctly initialize the HW for the LED status (on Arduino Opta expansions is present a RGB LED status which is used to signal the status of the expansion regarding the Address assignment: RED -> ready to get the I2C address, BLUE -> no address and waiting for other expansion to get their own address, GREEN -> I2C address acquired). In case Status led is not present just provide an empty, dummy implementation.
- ***virtual void setStatusLedReadyForAddress();*** this function must set the status LED in the state that indicates that the expansion has not I2C address but can acquire a new one. In case Status led is not present just provide an empty, dummy implementation.
- ***virtual void setStatusLedWaitingForAddress();*** this function must set the status LED in a state that indicates that the expansion has not address but is not ready to acquire new one (typically is waiting for other expansions to get their address). In case Status led is not present just provide an empty, dummy implementation.
- ***virtual void setStatusLedHasAddress();*** this function must set the status LED in a state that indicates that the expansion has acquired a valid I2C address. In case Status led is not present just provide an empty, dummy implementation.

3.3 Overriding Basic Virtual Methods

Basic Module class has 4 virtual methods that need to be overridden in your derived class.

- ***begin()*** must contain all the necessary initialization for the new expansion
- ***update()*** is the function that will be called repeatedly in your FW (see for example in the folder firmware for an example), this function should manage all the cyclical tasks that your application has to perform.
- ***end()*** must contain all the shutdown operation needed to properly stop the expansion
- ***parse_RX()*** will be called to give the opportunity to the expansion to recognize and parse the I2C messages received, in case the expansion recognize a message as to be handled it has to put in the TX buffer the correct message answer for that controller

request and **return the number of byte to be transmitted in the answer** (return -1 if the message is not handled by the expansion).

VERY IMPORTANT:

All previous overridden methods MUST always call the base version at the very beginning (this is because the base Module has to have the chance to perform all tasks common to every expansion). A note about ***parse_RX():*** when you call **Module::parse_RX()** in your overridden **parse_RX()** it will certainly happen that the base Module itself will be managing some messages, you don't need to deal with them, in this case just check for the returned value. If the value is -1 this means that the message has not been handled by the base version and you need to check if your expansion manages it, if the value is greater or equal to 0 the base Module version has already taken care of the message and you just need to return the same value.

VERY IMPORTANT:

Do not make any change to the ***expansion_type*** (an integer in the Module class). This is set in Module class and must NOT be modified in any Custom expansion.

3.3.1 Performance request

Each expansion performs Assign Address Process using a common class Module, which is contained in OptaBlueModule files (.h and .cpp). The Module::begin() function performs the assign address process at the very beginning, but the function Module::update() is responsible to change the I2C address of the expansion when a new expansion is plugged in and the assign address process is done a second time (this function has to be called in the NewExpansion::update() function, see above). In the latter case the change of the address (being performed in Module::update()) is done in the main loop of the expansion. So, there is a delay between the time the expansion gets the message (I2C interrupt) and the moment the expansion actually sets the address (in the main loop). The actual set of the expansion, in other words, depends on how "fast" is the main loop, however the main controller waits for a certain time and then retry the process. The expansion must set the new address as fast as it can to avoid retrieving from the controller. This means that it is strongly suggested that the main loop of the expansion (except for the call to the Module::update() function) only runs when the address is valid, so that when not a valid address has been set up, it can react very quickly to the address set request. A good way to understand if the expansion has a valid address is:

```
If ( wire_i2c_address <= OPTA_DEFAULT_SLAVE_I2C_ADDRESS ||
     wire_i2c_address >= OPTA_CONTROLLER_FIRST_TEMPORARY_ADDRESS) {
   // NO VALID ADDRESS ACQUIRED
}
```

3.4 OptaNewExpansionCfg.h

This is an optional configuration file that contains all the configuration ***#define*** used to configure FW. If the file is present, it must be included only in OptaNewExpansion.h file: this means that this file will contain information relevant only to the FW level and won't "export" any information useful at the **application** level (i.e. on the Opta Controller side).

3.5 Sharing information between FW and the controller

It might be good to have a place to put configurations or type definitions that are used both by FW and the Opta Controller application. We suggest calling this file NewExpansionCommonCfg.h (always remember to use the proper name of your expansion instead of 'NewExpansion' used here for simplicity).

This file will be included both in the OptaNewExpansion.h file (and this will inform the FW) and in the NewExpansionExpansion.h (that name is ugly but it simply means that the expansion class to be provided to the application / controller level is SomethingExpansion and since we decided to use 'NewExpansion' as the name of our made up new expansion the name of the file will NewExpansionExpansion.h)

So, at this point we can further populate our New Expansion library with the files to be used at the controller/application level.

/rood_folder_of_new_expansion_library

|-firmware/NewExpansion/NewExpansion.ino

|-src/

|-OptaNewExpansion.h

|-OptaNewExpansion.cpp

|-OptaNewExpansionCfg.h (optional)

|-NewExpansionCommonCfg.h (common share conf. between FW and application)

|-NewExpansionExpansion.h (header file for the expansion class at application level)

|-NewExpansionExpansion.cpp (implementation file for the expansion at the application level)

Just for completeness the following expresses the "allocation" of the files to FW or application.

/rood_folder_of_new_expansion_library

|-firmware/NewExpansion/NewExpansion.ino

|-src/

|-OptaNewExpansion.h (FW header class)

|-OptaNewExpansion.cpp (FW implementation class)

|-OptaNewExpansionCfg.h (FW configuration, optional)

|-NewExpansionCommonCfg.h (FW / APP shared configuration and types)

|-NewExpansionExpansion.h (APP header class)

|-NewExpansionExpansion.cpp (APP implementation class)

3.6 The NewExpansion Application Class

Class ***NewExpansionExpansion*** must be derived from Expansion class.

It is important to understand that Expansion class implements a very simple model valid for (hopefully) every expansion type. From the Expansion class perspective each expansion is just a bunch of "registers" (there are 2 kinds of registers, integer ***iregs*** and floating point ***fregs***). Those registers are implemented using ***maps*** so that you can dynamically add or remove registers on the fly. Each register is identified by an address that the user can define. For

example, have a look to DigitalExpansionAddress.h: the status of an output is represented by a register

- ADD_DIGITAL_0_OUTPUT represents the address of the register containing the status of the digital output 0,
- ADD_DIGITAL_1_OUTPUT represents the address of the register containing the status of the digital output 1 ...and so on. When you want to set the digital output 0 of the expansion to a high level you set 1 into the register at the address ADD_DIGITAL_0_OUTPUT and you send this to the expansion.

The Expansion class essentially provides 3 methods:

- read() to read a register
- write() to write a register
- execute() to execute an operation (such sending the new output status to the expansion to switch on an output).

Every operation performed on an expansion is implemented using these three basic functions.

To set up a programmable function channel for example:

1. Write a bunch of register holding the configuration of the channel
2. execute a ***configure***_(BEGIN_CHANNEL_AS_ADC), this will use the content of the proper registers to send a message to expansion (via the Controller class) that will inform the expansion about the desired configuration

In the same way you can get information from the expansion:

1. Write register holding information about what you want to get
2. execute a ***get*** operation (GET_SINGLE_ANALOG_OUTPUT)
3. read the registers that are filled by the execute function with the information you wanted to get from the expansion

This generic abstraction should work on every possible expansion.

Of course, read(), write() and execute() are virtual functions so that you can customize them for the needs of your expansion.

For example, to save memory and be more efficient the DigitalExpansion class overrides the standard Expansion::write() function. When you write a digital output register such ADD_DIGITAL_0_OUTPUT (for example) the overridden function will set a bit in the register at the address ADD_DIGITAL_OUTPUT. So that the register at the address ADD_DIGITAL_0_OUTPUT is not really allocated and does not take space in memory, moreover the register at the address ADD_DIGITAL_OUTPUT (that maps all the outputs as a bit field integer) is immediately ready to be transferred to the expansion to update the output during an execute.

3.6.1 About the execute() function

The execute() function has always the same structure (and we strongly suggest the same structure on each derived class): it takes an integer as parameter that identify the operation to be performed (set an output / read an input /configure a channel and so). Depending on the action to be executed the execute() function always performs an I2C transaction to communicate with the expansion. To this purpose I2C_TRANSACTION macro is used.

This macro (I2C_TRANSACTION) is conceived to be generic and takes 3 parameters:

- a pointer to a function preparing the message to be sent on I2C bus (this function returns the number of bytes to be transmitted)
- a pointer to a function able to parse the answer received by the controller (returns true if parsing is correct)
- the number of bytes expected in the answer

So, suppose we want to write an Arduino like ***digitalWrite()*** function for an expansion, these are the operation required:

1. the digitaWrite() function sets the registers holding the information about the output status
2. execute the operation SET_DIGITAL_OUTPUT
3. the execute() function "triggers" an I2C transaction using I2C_TRANSACTION macro
4. This function uses the "prepare" function pointer to understand how to set up the transmission buffer (this typically will write the information held in the registers into the transmission buffer)
5. send the transmission buffer using the Controlled function send()
6. wait for the controller to return an answer
7. parse the received answer using the pointer to the "parsing" function (this will write perhaps some other registers that can be read to return some information, although this is not the case of digitalWrite())

The I2C_TRANSACTION already takes care of timeouts and will call the failed communication callback if set.

IMPORTANT:

There are 3 operations the basic expansion class takes care which are common to all classes (WRITE in flash, READ in flash, get FW version). If you want to use these operations, remember to call ***Expansion::execute();*** in your ***default*** case switch of execute. This will use the base ***execute()*** function performing these basic operations.

IMPORTANT:

There is another important point to remember when overriding the execute() function (and this, of course is something every expansion must do in order to introduce specific operations): at the end of each ***execute*** function remember to call ***ctrl->updateRegs(*this);*** . Why this? Well, this library works using copies of expansion objects: you ask the controller to give you an expansion using OptaController.getExpansion() and the controller will return a copy of the proper expansion object that is held inside the controller. This is safe because you don't allocate anything and your copy will be destroyed as soon as it goes out of scope, however when you work on the expansion, **you are working *on a copy.*** This means that the content of the registers of the expansion held by the Controller are no longer aligned with the ones of your copy. Calling ***ctrl->updateRegs(*this);*** will copy back the content of your registers to the expansion object held by the controller, so that if you get another (perhaps in a different function) you don't lose any changes you made elsewhere. For example, look for a digital expansion execute function.

3.6.2 Mandatory functions to be implemented in NewExpansionExpansion class

In addition to the copy of i2c_transaction function, the following function must be always implemented in the NewExpansionExpansion class:

- a copy constructor in the form ***NewExpansionExpansion::NewExpansionExpansion(Expansion &other)*** This copy constructor is crucial. Simply copy an existing one and just change the types. If you look at the different copy constructors already implemented, they all do the same operation. They copy all the information from the object held by the controller into your object (also the registers), but only if the object held by the controller is of the same type (otherwise invalid expansion information is returned). This is important: you don't need to worry about allocating or deallocating object (this is managed by the controller), you always get a copy of your expansion that will be automatically destroyed when it goes out of scope. **IMPORTANT NOTE**: the copy constructor must use a special Controller function to know the type of the Expansion (see below section NewExpansion enumerative type).
- a static ***makeExpansion()*** method that returns an ***Expansion**** pointer and that actually allocates an object of the NewExpansionExpansion class via ***new*** C++ operator. This function will be used by the Controller to perform the allocation of the object in the correct way (this way the controller is agnostic of the expansions but can still make them). This function should not be called directly by an application, since it is intended that the Controller makes and deletes the objects. Besides, the use of this function to allocate an object would be completely meaningless because it will lack the necessary information that only the Controller can fill in.
- a static ***getProduct()*** method that return the string identifying the type of expansion. This function MUST return the exact same string returned by the getProduct() function defined at the FW level. If you look at the implementation of these two functions in the already implemented class, you'll see that they return the same string (which is a shared configuration parameter contained in the <Expansion>CommonCfg.h header file).
- a static ***StartUp(Controller *)*** method that will perform the initialization of your expansions (of all the expansions of the same type). The purpose of this function is very important, however hard to explain. Suppose that you have an expansion with some programmable output function (Opta Analog is a good example, because each channel can be RTD, ADC, DAC). The controller application will probably set up all the desired channel function once in the setup() function (the usual Arduino initialization function). So, the controller will execute the setup() function just once at the beginning (suppose setting channel 0 as DAC and channel 1 as ADC and so on). After that the Controller just uses the expansion and has no need to configure the channels of the expansion anymore. Now suppose that you must power down your expansion, but you don't want to reset also the controller (maybe because the controller is controlling other expansions you don't want to shut down). After you perform the operation on shut down expansion you power it up again. The Controller sees the "new" expansion, performs a new expansion discovery process and assign address and all work seamless, but... Well, the expansion you shut down lost its configuration, but the Controller did not perform again an initial setup() function because it was not reset, so the configuration is lost. The expansion, in this case, is not more usable until you reset the controller and the setup() function is performed again. The purpose of the static ***StartUp(Controller *)*** function is to avoid cases like that: this function must provide a way to reinitialize all the possible expansions of that type to the desired configuration. This can be a complex task, in the Analog expansion a specific class to hold configuration information for each possible Analog expansion is implemented in AnalogExpansionCfg.h file. This

function will be passed to the controller so that controller can re-initialize the expansions every time a discovery expansion process is finished. The StartUp function is always called after the assign address process is completed. Each StartUp callback must set up properly all the expansion of a certain type in 2 possible situations:

- When the controller is re-programmed (since single expansions are not reset) the StartUp must send to all the expansions all that is needed to put the expansion in its "default state" (output off and so on...)
- When a single expansion is hot-plugged into the chain (or one of the expansions is reset for some reason) the StartUp function must send to all the expansions of that type, all that is needed to put the expansion in last known state. This is the approach used with digital and Analog expansion, however the behavior of a StartUp function could depend on how expansion FW works: if your custom expansion type does not need such function just write an empty function.

Then the new expansion class can define whatever custom additional function: the important point here is to remain to read / write / execute paradigm.

3.7 Controller additional initialization function for Custom expansion

If you have followed the previous indications, you should have a FW and an application expansion class.Typically for "Arduino" expansion the processes to write sketches for OptaController that uses Arduino expansion (for example Opta Digital and Opta Analog) is quite straightforward:

1. call ***OptaController.begin()*** during ***setup():*** this will properly initialize the controller
2. if needed properly initialize the expansions you want to use with the right configuration (this can be performed in various ways, please have look at the example provided with this library for more details)
3. wherever you need ask the controller to give you a copy of an expansion using OptaController.getExpansion(index_of_expansion)
4. check for the validity of expansion using the bool() operator: the expansion will be valid only if the type of expansion you requested is the same as the "hardware" level
5. Use the expansion (set output, get input and so on)
6. as soon as your expansion copy goes out of scope it will be automatically deleted because it is a copy and is not allocated on the heap (so no worries for memory management, the controller will destroy the expansion object it holds when needed)

This should be quite simple (have a look at the provided example), however there is a missing point for custom expansion: the controller is not aware of the fact that additional custom expansion exists. How to deal with that?

VERY IMPORTANT:

When writing a sketch (an example) for a custom expansion (i.e. an expansion which is not a direct Arduino product) you must ***register*** the new expansion to the Controller using ***OptaController.registerCustomExpansion()*** just after the ***OptaController.begin()*** function. The ***OptaController.registerCustomExpansion()*** parameters are the function you added to your NewExpansionExpansion class: see previous paragraph.

So typically, you must perform a call like:

OptaController.registerCustomExpansion(NewExpansion::getProduct(),

NewExpansion::makeExpansion,

NewExpansion::StartUp);

Please note that I am assuming that your custom expansion is called ***NewExpansion*** but this is of course a generic name that has to be converted in the actual expansion name.

3.8 NewExpansion Enumerative Type

The use of ***registerCustomExpansion*** leads to an important point: custom expansions get a dynamically calculated "enumerative" type.

For example: Opta Analog is identified by a type number equal to EXPANSION_OPTA_ANALOG (the actual value is unimportant and should never be used, but in this case is 4). This means that if you use ***Controller.getExpansionType(i)*** and at the index ***I*** you have an Analog expansion you'll get always ***EXPANSION_OPTA_ANALOG*** value (i.e. 4).

This happens because the controller is aware of the existence of Analog expansion even if they do not call the register function (this is a privilege reserved for Arduino expansion).

But when you register a custom expansion via ***registerCustomExpansion*** this is not true anymore: the Controller registers the expansion and assigns to the expansion type a unique value.

This value is returned by the function r***egisterCustomExpansion*** itself (or -1 if something's wrong).

You don't need to save this value since it can be always retrieved using the controller function ***OptaController.getExpansionType(string)***. As the parameter of this function always use the NewExpansion::getProduct() function.

So, suppose that your custom expansion is at the position 2 in the chain of expansions, to know that the expansion is of New Expansion type you can do something like:

- to get the dynamical enumerative type assigned to your expansion type use ***OptaController.getExpansionType(NewExpansion::getProdut())***
- to get the dynamical actual enumerative type of expansion at the position 2 (this is related to the actual hardware) use **OptaController.getExpansionType(2)**

OptaController.getExpansionType(NewExpansion::getProdut()) returns the type of the product (dynamically calculate) equal for each product ***OptaController.getExpansionType(index)*** while the expression returns the actual type of the controller assigned to the custom expansion hardware.

Please note that the use of getExpansionType(index) is not so interesting at the practical level. You can always avoid using it by asking for an expansion using ***NewExpansionExpansion exp = OptaController.getExpansion(index***) and then checking for the validity of the returned expansion using the bool() operator (i.e. ***if(exp)*** -> this will be true only if the expansion at index is actually of type NewExpansion).

At this point we can understand the general form of the NewExpansion copy constructor:

NewExpansionExpansion::NewExpansionExpansion(Expansion &other) {

NewExpansionExpansion &de = (NewExpansionExpansion &)other;

```
  type = EXPANSION_NOT_VALID;
  i2c_address = 0;
  ctrl = other.getCtrl();
  index = 255;


  if (ctrl != nullptr) {
    if (other.getType() ==
ctrl->getExpansionType(NewExpansionExpansion::getProduct())) {
      iregs = de.iregs;
      fregs = de.fregs;
      type = other.getType();
      i2c_address = other.getI2CAddress();
      ctrl = other.getCtrl();
      index = other.getIndex();
    }
  }
}
```

IMPORTANT NOTE: The dynamical type is assigned by the Controller after the assign address process and the expansions are discovered. If you don't have any expansion NewExpansion attached and you use ***Controller.getExpansionType(NewExpansionExpansion::getProduct())*** you'll get -1 until a NewExpansion is attached to Opta (this ensure that you get only NewExpansionExpansion objects actually related with a real hardware).

3.9 About I2C Messages Definition

To communicate with expansion a protocol over I2C is defined.

For organization purposes the message "defines" have been spitted in different files for different expansions.

- OptaBlueProtocol.h contains a few helper generals defines to help in message definition (you should include this file in your message definition file)
- OptaModuleProtocol.h contains the definition of the messages that must be handled by every expansion. You don't have to worry about these messages, and you don't have to include this file in your expansion library, as far as you follow the previous instructions your expansion will manage correctly those messages.
- OptaDigitalProtocol.h contains all the messages related to Opta Digital
- OptaAnalogProtocol.h contains all the messages related to Opta Ana

Feel free to re-use already defined messages in your expansion (for example you may re-use message to set digital output defined for Opta Digital, as far as it fits your needs).

We suggest keeping the definition of your custom messages (i.e. specific messages handled by your custom expansion) in a separate header file (always to keep consistency with

OptaBluePrint library), so the final structure of your custom expansion library should be something like:

/rood_folder_of_new_expansion_library

|-firmware/NewExpansion/NewExpansion.ino

|-src/

|-OptaNewExpansion.h (FW header class)

|-OptaNewExpansion.cpp (FW implementation class)

|-OptaNewExpansionCfg.h (FW configuration, optional)

|-NewExpansionCommonCfg.h (FW / APP shared configuration and types)

|-NewExpansionExpansion.h (APP header class)

|-NewExpansionExpansion.cpp (APP implementation class)

|-OptaNewExpansionProtocol.h (I2C messages definition)

IMPORTANT:

The header file OptaNewExpansionProtocol.h must always include OptaBlueProtocol.h so that you can use same configuration and same message’s part definition.

OptaNewExpansionProtocol.h will be included both in application-level file (NewExpansionExpansion.h) and FW level file (OptaNewExpansion.h).

4 CUSTOM EXPANSION TYPE

Users of this library are free to implement its own expansion and use this library to support new expansion hardware.

To write a library for your new hardware compliant with this OptaBluePrint library please follow the chapter Add Custom Expansion

This chapter is intended to document and explains how "custom types" are created.

4.1 Expansion Type

Each Expansion is identified by a type: this is essentially a number identifying the type of hardware attached (so that the Opta controller can recognize different kinds of hardware).

Official Arduino expansions have a predefined type's number: for example, Arduino Analog expansion has ALWAYS type equal to EXPANSION_OPTA_ANALOG (which is 4).

Custom expansion does not have a fixed type of number, after all how can the OptaBluePrint library know about expansion that does not exist yet?

4.2 Register Expansion

When you use a custom expansion, you need (among other things) REGISTER your expansion so that OptaBluePrint library can know about that.

To do so you call: ***int registerCustomExpansion(std::string pr, makeExpansion_f f, StartUp_f su)*** the parameters are explained in Add Custom Expansion chapter (I won't repeat here).

The important point here is that this function always returns you're the "custom type" assigned to your custom expansion.

Your custom expansion now has this type assigned.

Pay attention: this type can be different depending on your application (in particular will depend on the number of custom expansions registered and the order they are registered), you must not rely on a particular value in your code because it is not guarantee that you will get always the same *number*.

IMPORTANT NOTE:

The best approach to register an expansion is to call ***registerCustomExpansion*** *before* the OptaController.begin() call. All works also if expansions are registered after the OptaController.begin() however this approach is slower because it requires the Controller to look for an expansion with no custom type associated.

4.3 How To Retrieve Register Expansion Type

To know what type has been assigned to your custom expansion use: ***int getExpansionType(std::string pr);*** this will retrieve the number assigned via the register function.In particular you can use something like that:***OptaController.getExpansionType(<your expansion type>::getProduct());*** since you have to define a "name" for you expansion type via the getProduct static function.

4.4 Different Overloads

Please note that ***int getExpansionType(std::string pr);*** and ***uint8_t getExpansionType(uint8_t i);*** function have a complete different meaning: the first one retrieve the "enumerative" number associated with a particular custom expansion, while the second one will return the type of the expansion present at the index i in the expansion chain (this is the same only if the expansion is really of "that" type).

An expression like this ***getExpansionType("MY CUSTOM EXPANSION") == getExpansionType(2)*** will be true only if you registered an expansion which has a product name equal to "MY CUSTOM EXPANSION" (choose a better name!) and in the position 2 there is actually and physically your custom expansion.

